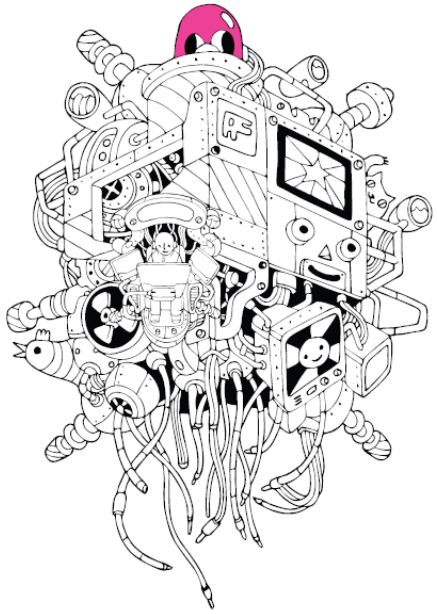
C programming



C Programming:

Chapter 03. Variables and Operators

C programming

Chapter 03-1. Operators for operations and Variables for storing values

C Programming:

+ operator required for implementing addition programs

```
int main(void)
{
    3+4;    // 3과 4의 합을 명령함
    return 0;
}
```

Nothing appears as a result of execution .

+

· It is certain that it is a recognizable symbol, as no problems occur during compilation and execution.

· In fact, + means addition. Therefore, the sum of 3 and 4 is performed through execution.

· Symbols such as + are called operators.

What is the result of the operation ?

· Only the + operation was requested, and no code was inserted to output the result.

· Therefore, no output is produced.

· You must save the results of the operation so that you can proceed further as desired.

· The C language provides something called a variable to store operation results or values.

Storing data using variables

√ What is a variable ?

A name given to a memory space where a value can be stored

When you declare a variable, memory space is allocated and a name is given to the allocated memory space.

√ Variable name

The allocated memory space can be accessed through the variable name.

You can store values or reference stored values.

```
int main(void)
{
    int num;
    num=20;
    printf("%d", num);
    . . . .
}
```

int num

- **Allocation of memory space for storing int integers**
- **The name of the allocated memory space is num**

num=20;

- **Access the variable num and store**

printf("%d", num);

Reference the value stored in num (output)

Various ways to declare and initialize variables

ch03_garbage.c

```
int main(void)
{
    int num1, num2;       // Declaration of variables num1 and num2
    int num3=30, num4=40;     // Declaration and initialization of variables num3 and num4

    printf("num1: %d, num2: %d \n", num1, num2);
    num1=10;      // Initialization of variable num1
    num2=20;      // Initialization of variable num2

    printf("num1: %d, num2: %d \n", num1, num2);
    printf("num3: %d, num4: %d \n", num3, num4);
    return 0;
}
```

Execution results

```
num1: -858993460, num2: -858993460
num1: 10, num2: 20
num3: 30, num4: 40
```

int num1, num2;

- **You can declare variables only without any value.**
- **You can declare two or more variables at the same time using commas.**
- **If you just declare it, it will be filled with garbage values (meaningless values) until a value is assigned.**

int num3=30, num4=40;

- **Can be initialized at the same time as declaration .**

Things to consider when declaring variables

```
int main(void)
{
    int num1;
    int num2;
    num1=0;
    num2=0;
    . . . .
}
```

Variable declaration that's compilable

```
int main(void)
{
    int num1;
    num1=0;
    int num2;
    num2=0;
    . . . .
}
```

Variable declarations that may prevent compilation

The old C standard required variable declarations to come first, and some compilers still follow that standard.

Variable naming rules **The most important thing is to give it a meaningful name!!**

1. Variable names consist of alphabets, numbers, and underscores (_).
2. The C language is case-sensitive. Therefore, the variables Num and num are different variables.
3. Variable names cannot start with numbers, and keywords cannot be used as variable names (keywords will be explained later).
4. No spaces can be inserted between names.

Wrong names!!

```
int 7ThVal;  // Because the variable name starts with a number
int phone#;  // Special characters such as # cannot be included in variable names.
int your name; // Variable names cannot contain spaces.0
```

Data Type

int num1 = 24

· num1 is an int type variable among integer variables

double num2 = 3.14

· num2 is a double type variable

√ Two types of integers

Integer and real number(floating-point) variables

√ Integer variable

A variable declared for the purpose of storing integer values

Integer variables are divided into char , short , int , and long types.

√ Real number(floating-point) variable

A variable declared for the purpose of storing real numbers.

Real-valued variables are divided into float-type variables and double-type variables.

√ Why are integer variables and real number variables separated ?

Because the way integers are stored and the way real numbers are stored are different.

Completing the addition program

```
int main(void)
{
    int num1=3;
    int num2=4;
    int result=num1+num2;
    printf("Addition result: %d \n", result);
    printf("%d+%d=%d \n", num1, num2, result);
    printf("The sum of %d and %d is %d.\n", num1, num2, result);
    return 0;
}
```

Execution results

```
Addition result: 7
3+4=7
The sum of 3 and 4 is 7.
```

Because we declared a variable to store the result of addition, we can output the result of addition in various forms.

C programming

Chapter 03-2. Various aspects of the C language

Introduction to operators

C Programming:

Assignment operators and arithmetic operators

Operator	Function of the Operator	Associativity Direction
=	Assigns the value on the right side of the operator to the variable on the left side. Example: num = 20;	←
+	Adds the values of the two operands. Example: num = 4 + 3;	→
-	Subtracts the value of the right operand from the value of the left operand. Example: num = 4 - 3;	→
*	Multiplies the values of the two operands. Example: num = 4 * 3;	→
/	Divides the value of the left operand by the value of the right operand. Example: num = 7 / 3;	→
%	Returns the remainder when the left operand is divided by the right operand. Example: num = 7 % 3;	→

```
int main(void)
{
    int num1=9, num2=2;
    printf("%d+%d=%d \n", num1, num2, num1+num2);
    printf("%d−%d=%d \n", num1, num2, num1-num2);
    printf("%d×%d=%d \n", num1, num2, num1*num2);
    printf("%d÷%d quotient=%d \n", num1, num2, num1/num2);
    printf("Remainder of %d%d=%d \n", num1, num2, num1%num2);
    return 0;
}
```

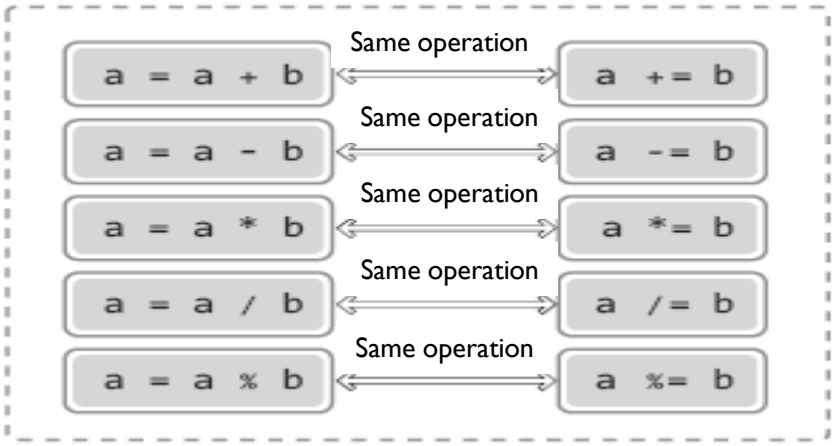
If there is an arithmetic expression in the function call statement, An operation is performed and a function is called based on the result.

Ex) Function (10 + 20)

Execution results

```
9+2=11
9−2=7
9×2=18
Quotient of 9÷2 = 4
Remainder of 9÷2 = 1
```

Compound assignment operators




```
int main(void)
{
    int num1=2, num2=4, num3=6;
    num1 += 3;    // num1 = num1 + 3;
    num2 *= 4;    // num2 = num2 * 4;
    num3 %= 5;    // num3 = num3 % 5;
    printf("Result: %d, %d, %d \n", num1, num2, num3);
    return 0;
}
```

Execution results

```
Result: 5, 16, 1
```

+ and – operators with sign meaning

```
int main(void)
{
    int num1 = +2;
    int num2 = -4;

    num1 = -num1;
    printf("num1: %d \n", num1);
    num2 = -num2;
    printf("num2: %d \n", num2);
    return 0;
}
```

int num1 = 2; and Same sentence!

Compilation is possible because + is included in the category of operators.

Execution results

```
num1: -2
num2: 4
```

Be careful not to confuse the two operators.

```
num1=-num2;      // Use of sign operators
num1-=num2;      // Use of compound assignment operators
```

Spacing to minimize confusion

```
num1 = -num2;    // Use of sign operators
num1 -= num2;    // Use of compound assignment operators
```

Increment and decrement operators

Operator	Function of the Operator	Associativity Direction
++num	Increases the value by 1 first, then executes the rest of the expression (prefix increment). Example: val = ++num;	←
num++	Executes the expression first, then increases the value by 1 (postfix increment). Example: val = num++;	←
--num	Decreases the value by 1 first, then executes the rest of the expression (prefix decrement). Example: val = --num;	←
num--	Executes the expression first, then decreases the value by 1 (postfix decrement). Example: val = num--;	←

```
int main(void)
{
    int num1=12;
    int num2=12;
    printf("num1: %d \n", num1);
    printf("num1++: %d \n", num1++);
    printf("num1: %d \n\n", num1);
    printf("num2: %d \n", num2);
    printf("++num2: %d \n", ++num2);
    printf("num2: %d \n", num2);
    return 0;
}
```

postfix increment

prefix increment

```
num1: 12
num1++: 12
num1: 13

num2: 12
++num2: 13
num2: 13
```

Execution results

Additional examples of increment & decrement operators

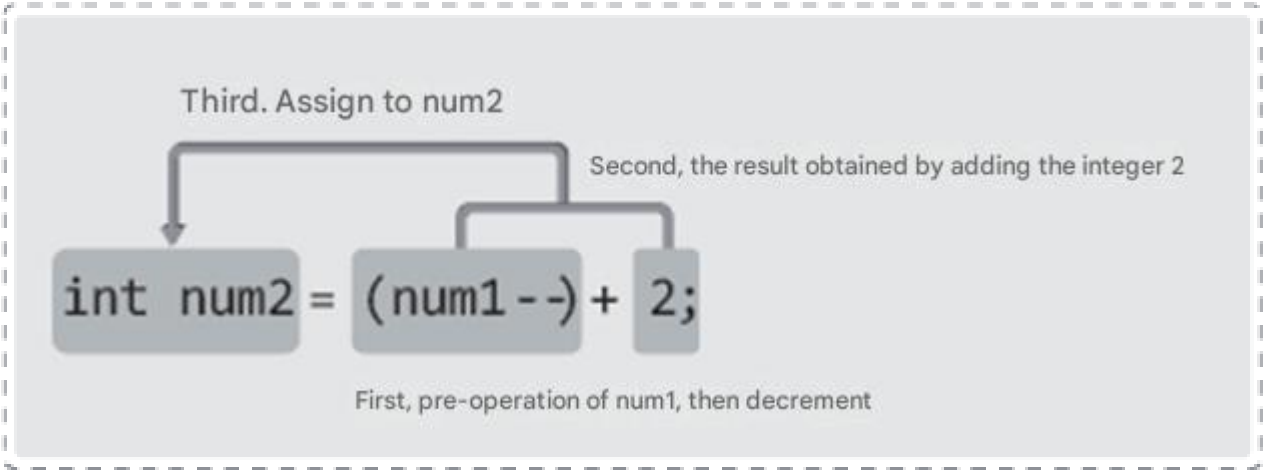
```
int main(void)
{
    int num1=10;
    int num2=(num1--)+2;    // 
    printf("num1: %d \n", num1);
    printf("num2: %d \n", num2);
    return 0;
}
```

postfix decrement

```
num1: 9
num2: 12
```

Execution results

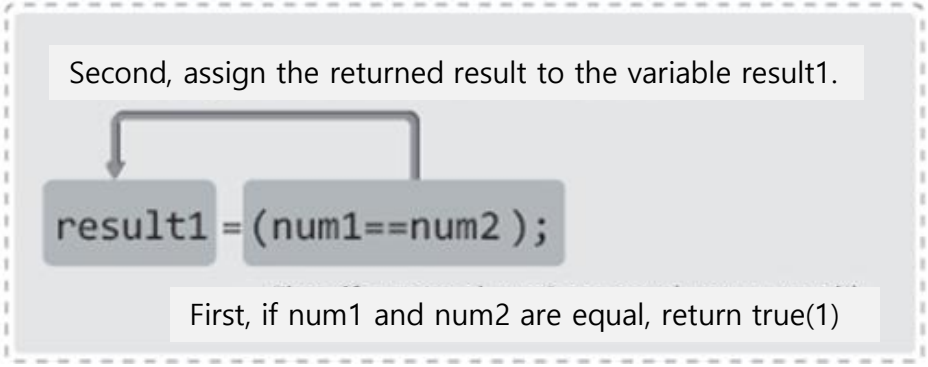
The process of operation



Relational operators

Operator	Function of the Operator	Direction
<	Example: n1 < n2 Is **n1 less than n2**?	→
>	Example: n1 > n2 Is **n1 greater than n2**?	→
==	Example: n1 == n2 Are **n1 and n2 equal**?	→
!=	Example: n1 != n2 Are **n1 and n2 not equal**?	→
<=	Example: n1 <= n2 Is **n1 less than or equal to n2**?	→
>=	Example: n1 >= n2 Is **n1 greater than or equal to n2**?	→

These are operators that return 1, meaning true, if the operation condition is satisfied, and 0, meaning false, if not satisfied.



C language considers any non- zero value to be true, although 1 is just a representative value that means true.

```
int main(void)
{
    int num1=10;
    int num2=12;
    int result1, result2, result3;
    result1=(num1==num2);
    result2=(num1<=num2);
    result3=(num1>num2);
    printf("result1: %d \n", result1);
    printf("result2: %d \n", result2);
    printf("result3: %d \n", result3);
    return 0;
}
```

Execution results

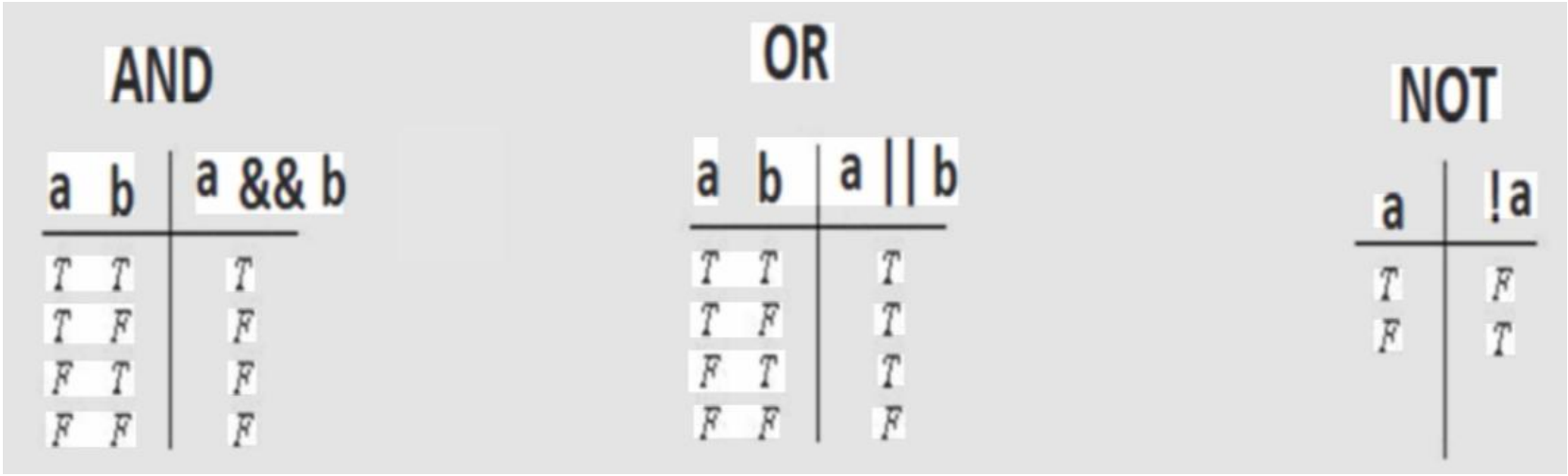
```
result1: 0
result2: 1
result3: 0
```

Logical operators

Common Logical Operators in C

- && → AND (true if both conditions are true)
- || → OR (true if at least one condition is true)
- ! → NOT (reverses the condition)

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT



Logical operators

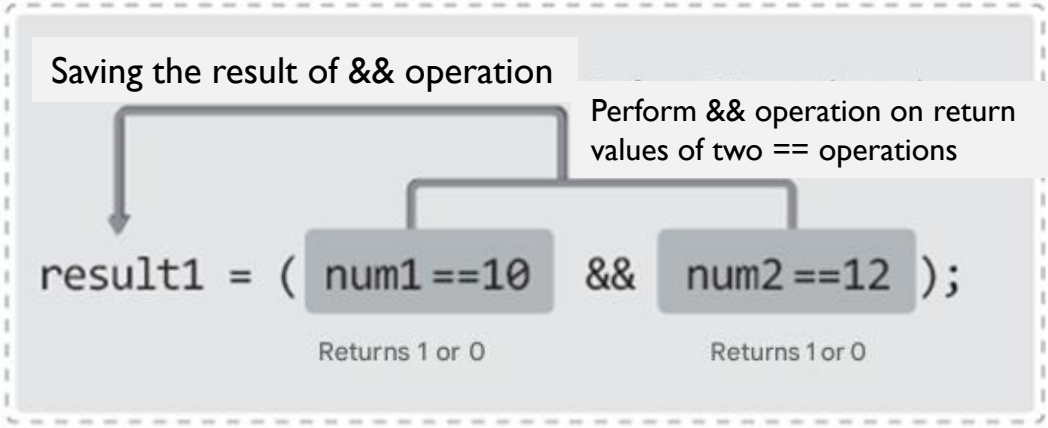
ch03_op.c

Operator	Function of the Operator	Associativity Direction
&&	Example: A && B → Returns **true** if **both A and B are true** (Logical AND).	→
||	Example: A || B → Returns **true** if **either A or B is true** (Logical OR).	`
!	Example: !A → If **A is true**, returns **false**; if **A is false**, returns **true** (Logical NOT)	←

```
int main(void)
{
    int num1=10;
    int num2=12;
    int result1, result2, result3;

    result1 = (num1==10 && num2==12);
    result2 = (num1<12 || num2>12);
    result3 = (!num1);

    printf("result1: %d \n", result1);
    printf("result2: %d \n", result2);
    printf("result3: %d \n", result3);
    return 0;
}
```



In the example on the left, num1 is not 0, so it is false in terms of the relationship between true and false. Therefore, the result of the ! operation is 1, which means true.

```
result1: 1
result2: 1
result3: 0
```

Execution results

Comma operator

√ comma (,)

· Comma is also an operator.

· This is an operator used when declaring two or more variables at the same time or inserting two or more statements in one line.

· It is also used to distinguish arguments when passing two or more arguments to a function.

· Unlike other operators, the comma operator is intended for ' separation ' rather than the result of the operation.

```
int main(void)
{
    int num1=1, num2=2;
    printf("Hello "), printf("world! \n");
    num1++, num2++;
    printf("%d ", num1), printf("%d ", num2), printf("\n");
    return 0;
}
```

Execution results

```
Hello world!
2 3
```

Operator precedence and associativity

√ Operator precedence

· Ranking of the order of operations

· Multiplication and division have higher priority than addition and subtraction.

√ Direction of association of operator

· Direction of operation between two operators with the same priority

· Addition, subtraction, multiplication, and division all proceed from left to right.

```
3 + 4 * 5 / 2 – 10
```

Multiplication and division are performed first based on operator precedence.

Multiplication occurs before division based on the direction of combination.

Operator precedence and associativity can be confusing.
Tip: Use parentheses or split the expression into multiple statements.

Operator precedence and associativity

√ Operator Precedence and Associativity

• Operator precedence determines which operator is evaluated first.

• Associativity determines the order when operators have the same precedence.

Examples:

*** has higher precedence than +**

→ a + b * c = a + (b * c)

/ and * have the same precedence

→ evaluated from left to right

→ 100 / 10 * 2 = (100 / 10) * 2

Assignment operators (=) are right-to-left

→ a = b = c = 10

→ a = (b = (c = 10))

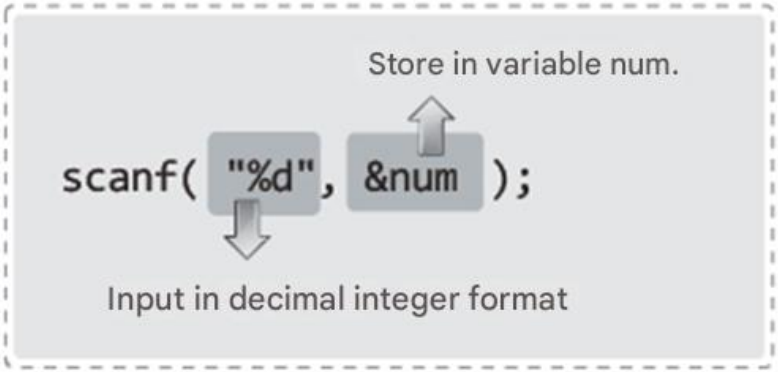
C programming

Chapter 03-3. Data input from the keyboard and C language keywords

C Programming:

Calling the scanf function to input integers from the keyboard

```
int main(void)
{
    int num;
    scanf("%d", &num);
    . . . .
}
```



- %d in the printf function means outputting a decimal integer.
- On the other hand , %d in the scanf function means input of a decimal integer
- The reason for putting & in front of the variable name num will be revealed later

```
int main(void)
{
    int result;
    int num1, num2;
    printf("Integer one: ");
    scanf("%d", &num1);      // input first integer
    printf("Integer two: ");
    scanf("%d", &num2);      // input second integer
    result=num1+num2;
    printf("%d + %d = %d \n", num1, num2, result);
    return 0;
}
```

printf only needs the value, but
scanf requires a space to store the value.

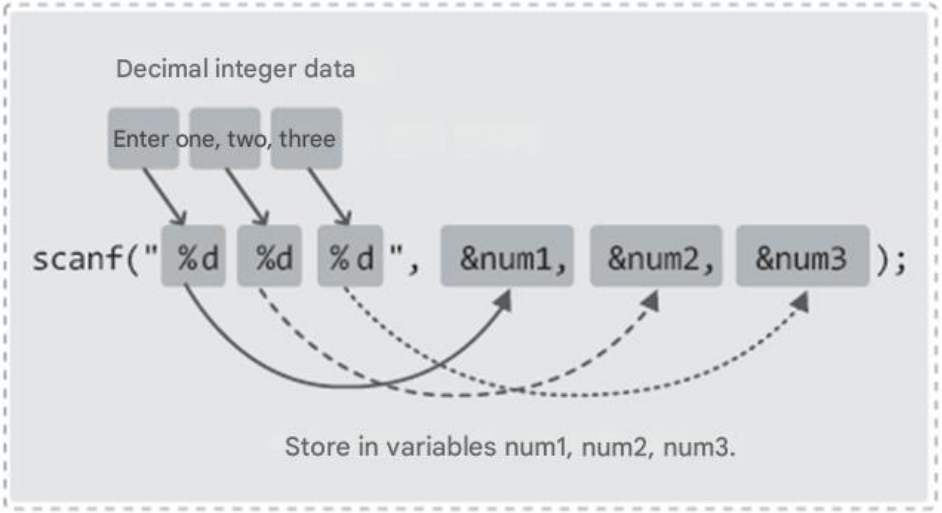
Execution results

```
integer one: 3
integer two: 4
3 + 4 = 7
```

practice

You can specify various input formats .

```
int main(void)
{
    int num1, num2, num3;
    scanf("%d %d %d", &num1, &num2, &num3);
    . . . .
}
```



You can input multiple data sets in any desired form at with a single scanf function call..

```
int main(void)
{
    int result;
    int num1, num2, num3;
    printf("Enter three integers: ");
    scanf("%d %d %d", &num1, &num2, &num3);
    result=num1+num2+num3;
    printf("%d + %d + %d = %d \n", num1, num2, num3, result);
    return 0;
}
```

Execution results

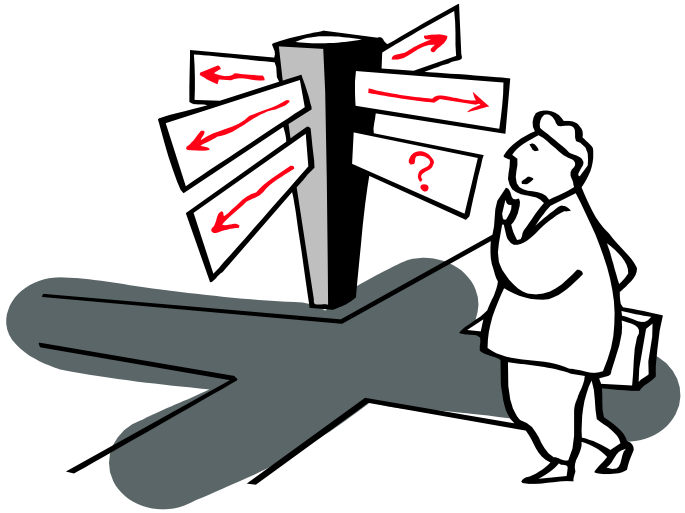
```
Input three integers: 4 5 6
4 + 5 + 6 = 15
```

Standard keywords of the C language

```
auto        _Bool       break       case
char        _Complex    const       continue
default     do          double      else
enum        extern      float       for
goto        if          _Imaginary  return
restrict    short       signed      sizeof
static      struct      switch      typedef
union       unsigned    void        volatile
while
```

Words with defined meanings that make up the grammar of the C language!

These words are called keywords.



Chapter 03 This That's it . Do you h
ave any questions ?